

Entwurf digitaler Systeme

8. Endliche Zustandsautomaten

Finite State Machines (FSM)

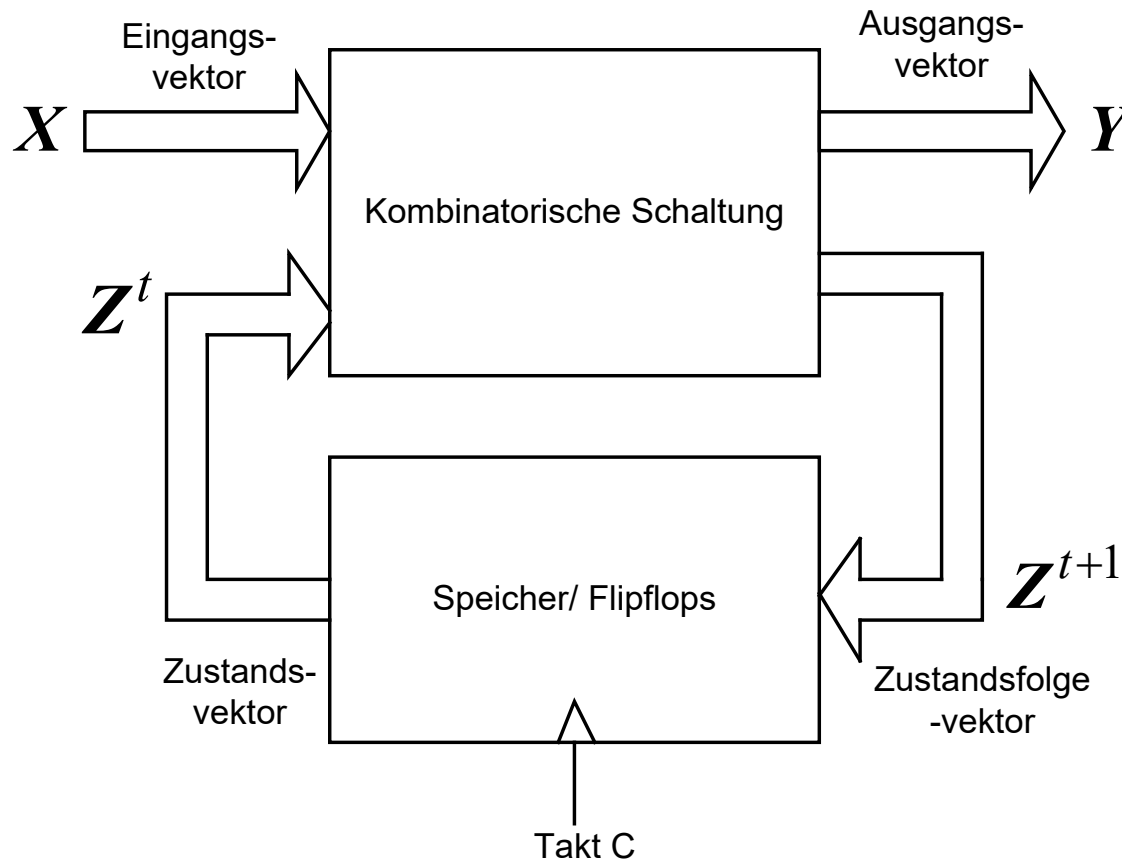
8. Endliche Zustandsautomaten

- Neben den autonomen Automaten (Zähler, Teiler, Sequenzer) werden in digitalen Systemen vielfach so genannte Endliche Zustandsautomaten (Finite State Machine, FSM) verwendet. Sie dienen zur Ablaufsteuerung von technischen Vorgängen in Abhängigkeit verschiedener Eingangsgrößen (Vektoren, Bündeln, Matrizen).

- **Definition 8.1** (Endlicher Zustandsautomat)
 - Ein Endlicher Zustandsautomat ist ein allgemeines sequentielles System, dass zur Steuerung von technischen Vorgängen dient. Es ist abhängig von seinen äußeren, von dem Prozess abhängigen, Zuständen sowie seinen inneren Folgezuständen.

8. Endliche Zustandsautomaten

- Allgemeines Blockschaltbild



8. Endliche Zustandsautomaten

- Unter praktischen Gesichtspunkten haben sich drei Automaten-Typen herausgebildet, die einsetzbar und entwerfbar sind. Sie sind in der Reihenfolge ihrer Komplexität aufgeführt.
 - Medvedjev-Automat
 - Moore-Automat
 - Mealy-Automat
- Wir werden diese Typen später genauer beschreiben. Zunächst sollen einige Grundlagen gelegt werden.

8.1 Grundlagen der Endlichen Zustandsautomaten

- Bei FSMs müssen alle Eingangs-, Zustands- und Ausgangsvariablen als Matrizen geschrieben werden, da nicht nur verschiedene Eingänge, sondern auch verschiedene Belegungen zu unterschiedlichen Zeiten auftreten.
- Wir wollen von der doppelten Indizierung für jede Variable absehen und die Variablen vereinfacht aufschreiben:
 - Es gelten die folgenden Formulierungen für einen Zeitpunkt t . Natürlich treten weitere Vektoren für die Zeitpunkte $t+1$, $t+2$, usw. auf. Die temporäre Abhängigkeit soll implizit angenommen und nur durch die symbolische Indizierung mit t und $t+1$ anfänglich dargestellt werden.

8.1 Grundlagen der Endlichen Zustandsautomaten

- Die Eingaben in einen Automaten werden mit Hilfe eines Eingangsvektors oder -bündels beschrieben.

$$\mathbf{X} = (x_{N-1}, x_{N-2}, \dots, x_0), \quad N \in G$$

- Der Eingangsvektor veranlasst den Automaten zum gegenwärtigen Zustand – dem Momentanzustand – einen definierten Ausgangszustand einzunehmen.

$$\mathbf{Z} = (z_{M-1}, z_{M-2}, \dots, z_0), \quad M \in G$$

- Momentanzustand: $\mathbf{Z} = \mathbf{Z}^t$

8.1 Grundlagen der Endlichen Zustandsautomaten

- Dieser Ausgangszustand ist nicht nur von dem Eingangsvektor abhängig, sondern auch von seiner Vorgeschichte. Damit ist der Ausgangsvektor

$$Y = (y_{K-1}, y_{K-2}, \dots, y_0), \quad K \in G$$

vom *Eingangsvektor* und dem *Momentanvektor* abhängig. Der neue Zustand, der sich durch die entsprechende Änderung ergibt, wird als Folgezustand

$$Z^* = Z^{t+1}$$

bezeichnet.

- Folgezustand: $Z^* = (z_{M-1}, z_{M-2}, \dots, z_0), \quad M \in G$

8.1 Grundlagen der Endlichen Zustandsautomaten

- Der Folgezustand wird mit Hilfe einer Übergangsfunktion f bestimmt. Hierbei sind prinzipiell verschiedene Übergangsfunktionen denkbar.
- Für den Automaten gilt $Z^* = f(X, Z)$.
- *Zum Vergleich:* Zähler $\rightarrow Z^* = f(Z)$
- Ausgangsfunktion FSM: $Y = f(X, Z)$
- *Zum Vergleich:* Zähler $\rightarrow Y = f(Z)$

8.1 Grundlagen der Endlichen Zustandsautomaten

- Automaten werden in zwei Hauptgruppen klassifiziert:
 - Asynchrone Automaten,
 - Synchrone Automaten.
- Die Realisierung der Rückführung entscheidet über den Automatentyp. Asynchrone Automaten besitzen ein Schaltnetz als Rückführung. Mit jeder Änderung von X und Z erfolgt ein sofortiger Zustandsübergang. Asynchrone Automaten besitzen die Eigenschaft der *virtuellen Instabilität*. Bedingt durch die kombinatorische Rückführung können während Zustandsänderungen Mitkopplungen auftreten. Der Automat beginnt unkontrolliert zu „schwingen“ und erreicht gegebenenfalls keinen stabilen Zustand.

8.1 Grundlagen der Endlichen Zustandsautomaten

- Das erwähnte instabile Verhalten lässt sich durch geeignete Codierungsverfahren der Zustandsvariablen minimieren. Optimal sind Codierungen, die sich bei jeder Zustandsänderung nur in einer Variablen ändern. Weit verbreitet ist in diesem Zusammenhang der uns bereits vertraute *Gray-Code*. Dieser wird unter praktischen Gesichtspunkten gern verwendet. Es sei darauf hingewiesen, dass weitere Möglichkeiten bestehen, das oben genannte störende Verhalten zu reduzieren (*One-Hot*). Hier muss jedoch auf die weiterführende Literatur verwiesen werden.

8.1 Grundlagen der Endlichen Zustandsautomaten

- *Synchrone* Automaten besitzen Speicherelemente (z.B. flankengesteuerte D-FFs) als Rückführung. Der Zustandsübergang wird hier mit der Taktzustandsänderung synchronisiert. Die Randbedingung ist hierbei, dass die Zustandsänderung *taktsynchron* zu erfolgen hat.
- **Goldene Regel**
 - Verwenden Sie in der Praxis – unter allen Umständen – synchrone Automaten, da sie einfacher zu entwerfen sind und sich die Kopplung untereinander einfach gestalten lässt.

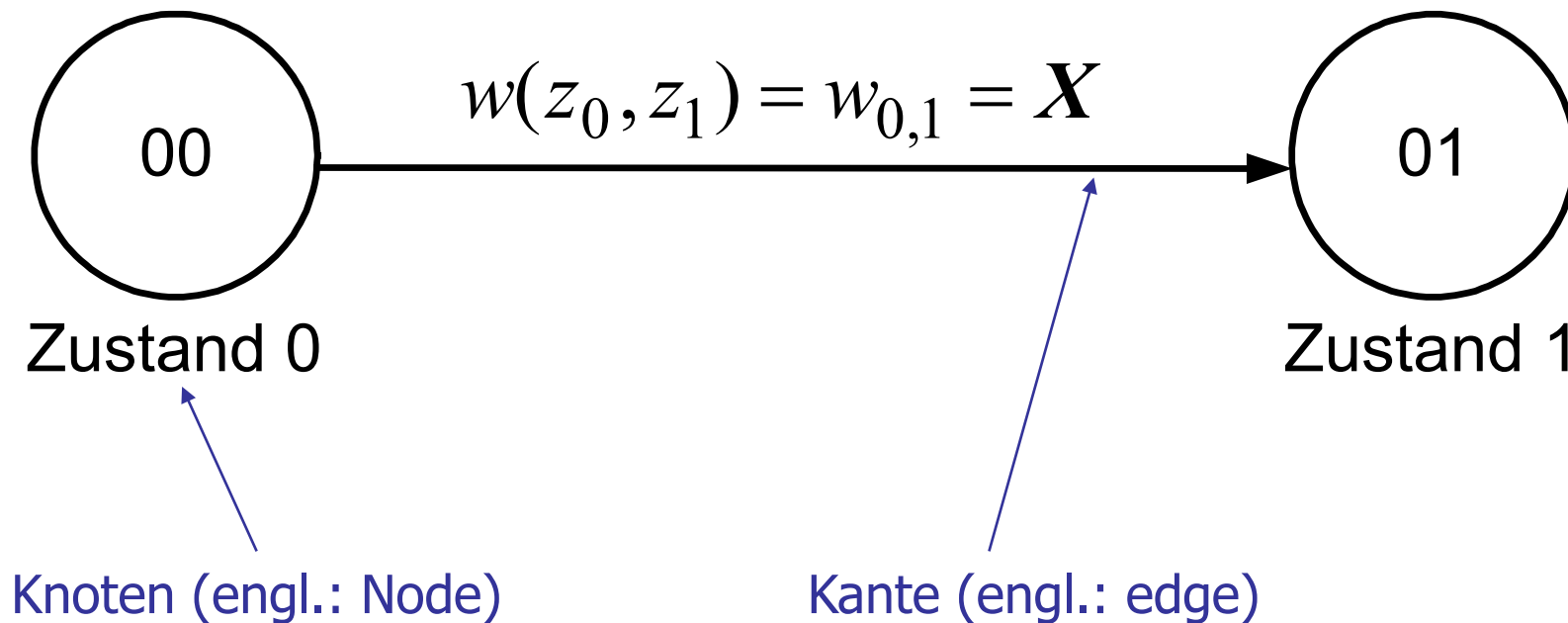
8.1 Grundlagen der Endlichen Zustandsautomaten

- Das Verhalten von Automaten lässt sich mit Hilfe eines graphischen und eines tabellarischen Schemas darstellen und entwerfen. Die Bezeichnungen für diese Schemata lauten:
 - Automatengraph und
 - Zustandsfolgetabelle.
- Beide haben wir bereits bei dem systematischen Entwurf von Zählern kennengelernt. Eine Vertiefung der Zusammenhänge erfolgt im nächsten Abschnitt.

8.1.1 Automatengraph und Zustandsfolgetabelle

- Automatengraph

Gewicht (engl.: weight)



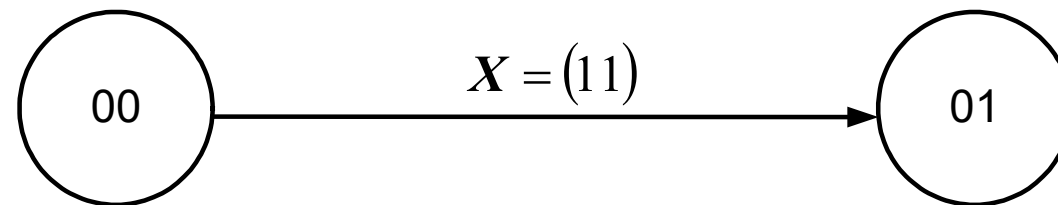
8.1.1 Automatengraph und Zustandsfolgetabelle

■ Übergänge

Wenn

$X = (11)$

bedingter Übergang

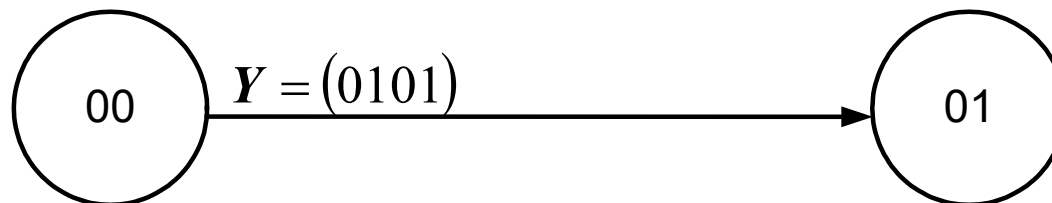


dann

Übergang $00 \rightarrow 01$

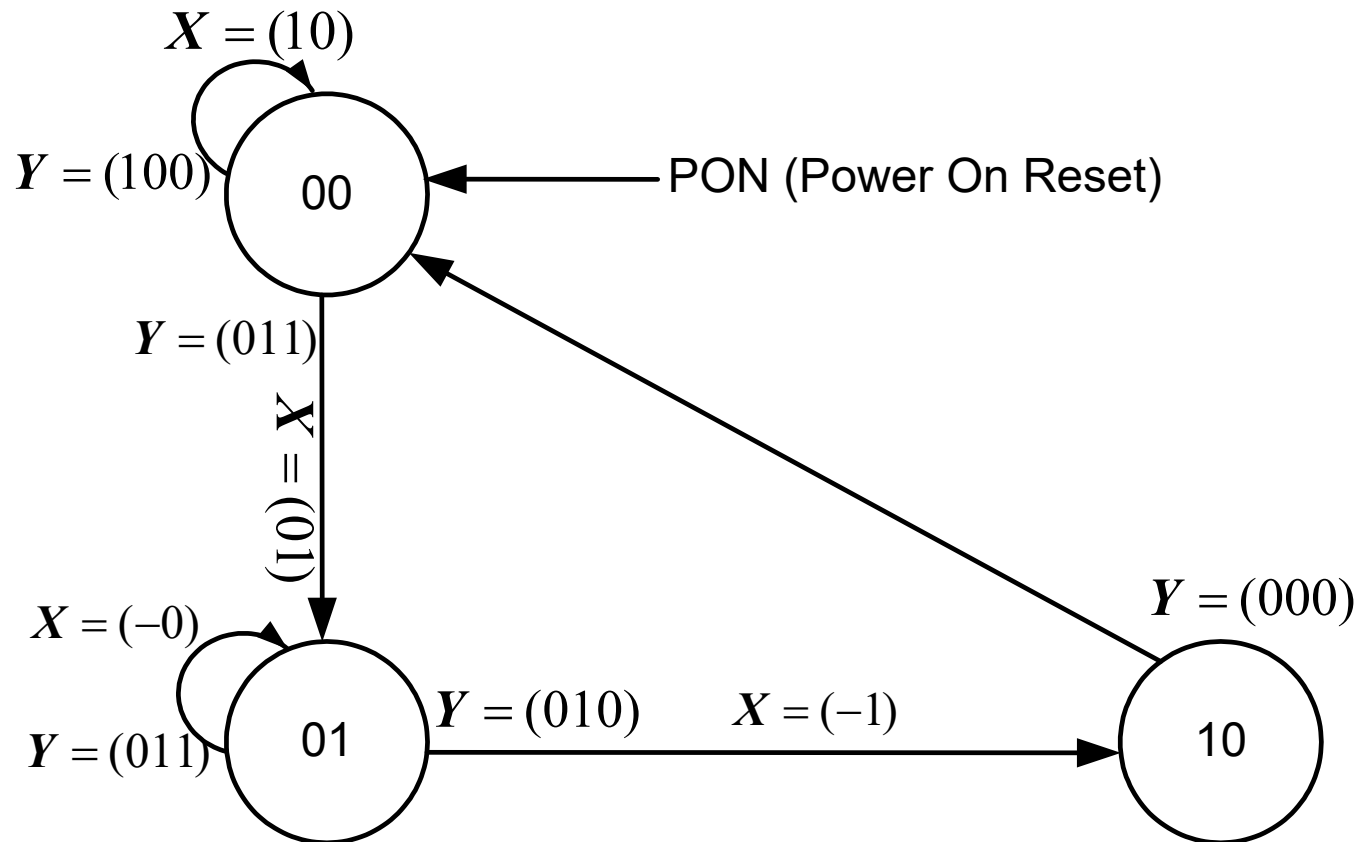
$Y = (0101)$

unbedingter Übergang



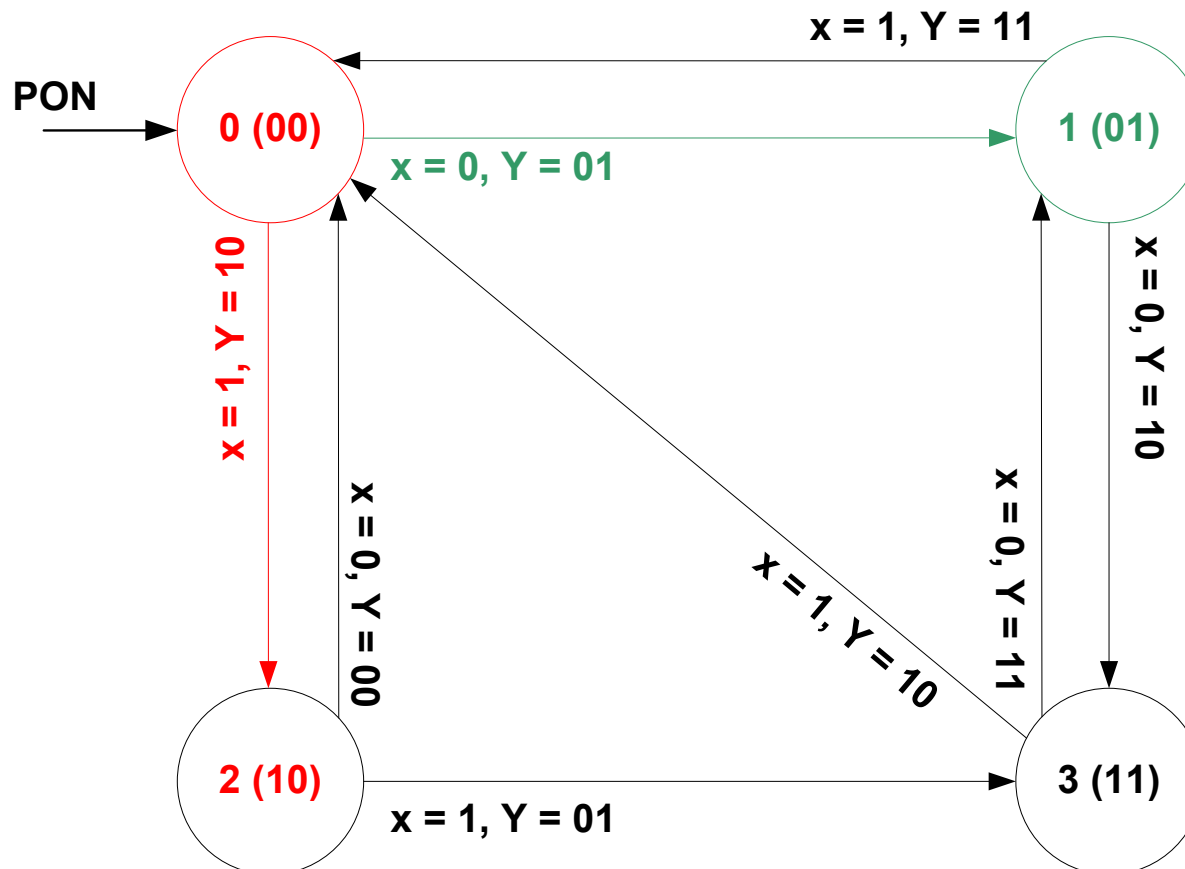
8.1.1 Automatengraph und Zustandsfolgetabelle

- Beispiel: Automatengraph



8.1.1 Automatengraph und Zustandsfolgetabelle

- Beispiel zur Tabelle (n. Seite)



8.1.1 Automatengraph und Zustandsfolgetabelle

- Automaten lassen sich mit der Hilfe von *Zustandsfolgetabellen* entwerfen. In einer Zustandsfolgetabelle werden alle Variablen der Vektoren X , Z , Z^* , und Y in entsprechende Spalten eingetragen. Beginnend mit dem Zustand 0, dem *Anfangszustand*, werden für alle vorkommenden Kombinationen der Eingangsvariablen alle Folge- und Ausgangszustände bestimmt. Anschließend wird mit dem Folgezuständen fortgefahren.

X	Z	Z*	Y
$x = x_0$	$z_1 z_0$	$z_1^* z_0^*$	$y_1 y_0$
0 1	0 0 0 0	0 1 1 0	0 1 1 0
0 1	0 1 0 1	1 1 0 0	1 0 1 1
0 1	1 0 1 0	0 0 1 1	0 0 0 1
0 1	1 1 1 1	0 1 0 0	1 1 1 0

8.1.1 Automatengraph und Zustandsfolgetabelle

- Wie Sie sehen, werden für den Eingangsvektor – in diesem Fall ist er zu einem Skalar degeneriert – jeweils alle Kombinationen der Zustandsvariablen, ihrer Folgezustände und der damit verbundenen Ausgangsbelegungen notiert. Besteht der Eingangsvektor aus N Vektoreinträgen, der Zustandsvektor aus M Einträgen, werden in der Zustandsfolgetabelle

$$\xi = 2^{N+M}$$

Zeilen *maximal vorgehalten*, die die Zustandsübergänge und deren Charakterisierungen beschreiben. Im oben genannten Beispiel ergeben sich somit 8 Zeilen. Sie werden feststellen, dass diese Anzahl in vielen Fällen nicht benötigt wird. Wir wollen den Sachverhalt an einem weiteren Beispiel vertiefen.

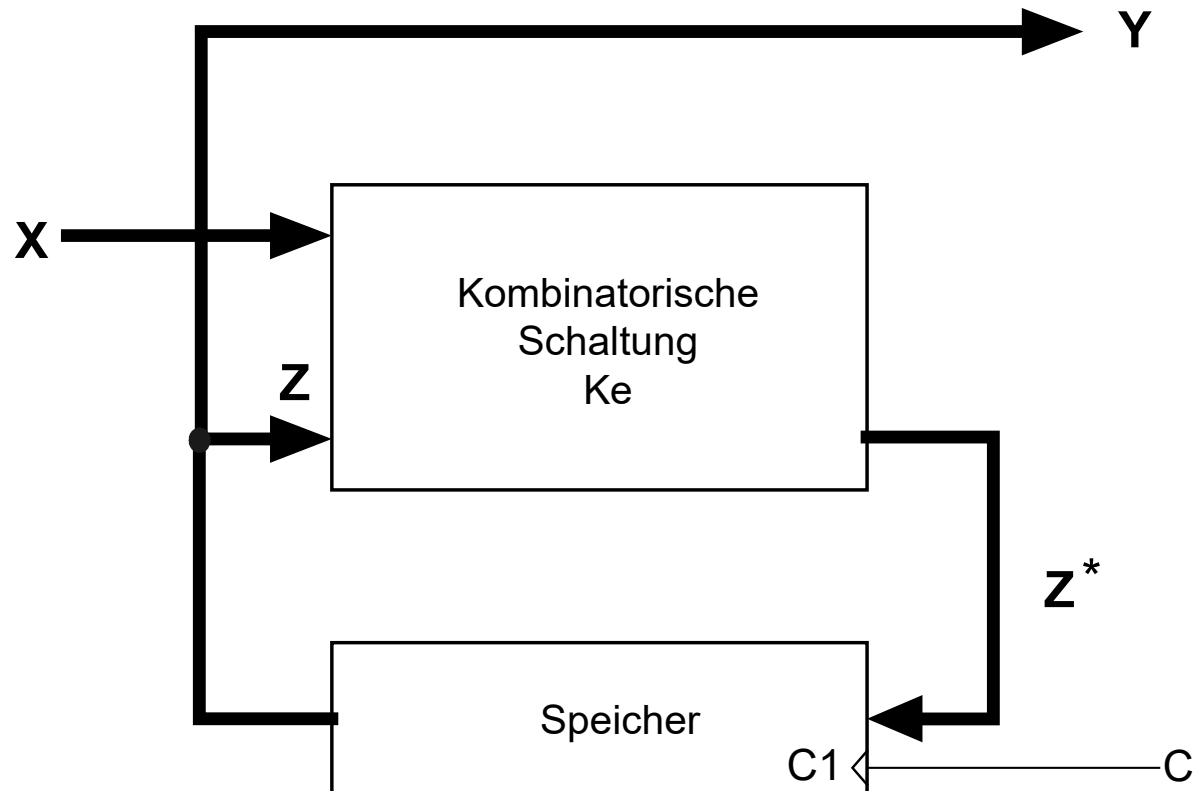
8.2 Medvedjev-Automat

- Der *Medvedjev-Automat* ist der einfachste synchrone Automat. Er stellt einen Sonderfall des bekannteren *Moore-Automaten* dar. Deshalb wird dieser Automat nur kurz beschrieben. Alle weiteren Prinzipien werden am Moore-Automat erläutert.
- Der Medvedjev-Automat wird durch seine charakteristische Übergangs- und Ausgangsfunktion beschrieben.
- Die Übergangsfunktion ist von den momentanen Werten der Eingangs- und Zustandsvariablen abhängig. Der Ausgangsvektor ist identisch mit dem Zustandsvektor.

$$\mathbf{Z}^* = f(\mathbf{X}, \mathbf{Z}) \quad \mathbf{Y}^t = \mathbf{Z}^t$$

8.2 Medvedjev-Automat

- Signalflussgraph bzw. Blockdiagramm



8.2 Medvedjev-Automat

- Der Entwurf eines Medvedjev-Automaten soll anhand eines Beispiels erläutert werden. Er unterscheidet sich nicht wesentlich vom Entwurf eines synchronen Zählers.
- Beispiel: Pulslängen-Zähler
 - Ein Medvedjev-Automat soll das Auftreten des Eingangssignals $\mathbf{X} = (x, R)$ überwachen, das die Länge von 0 bis 3 Taktimpulsen annehmen kann. Ist das Eingangssignal für mindestens 3 Taktimpulse für eine beliebige Zeit aktiv gewesen ($x = 1$), gibt der Medvedjev-Automat diesen Wert aus und wartet auf ein Reset-Signal ($R = 1$) um in seinen Grundzustand zurückzukehren.

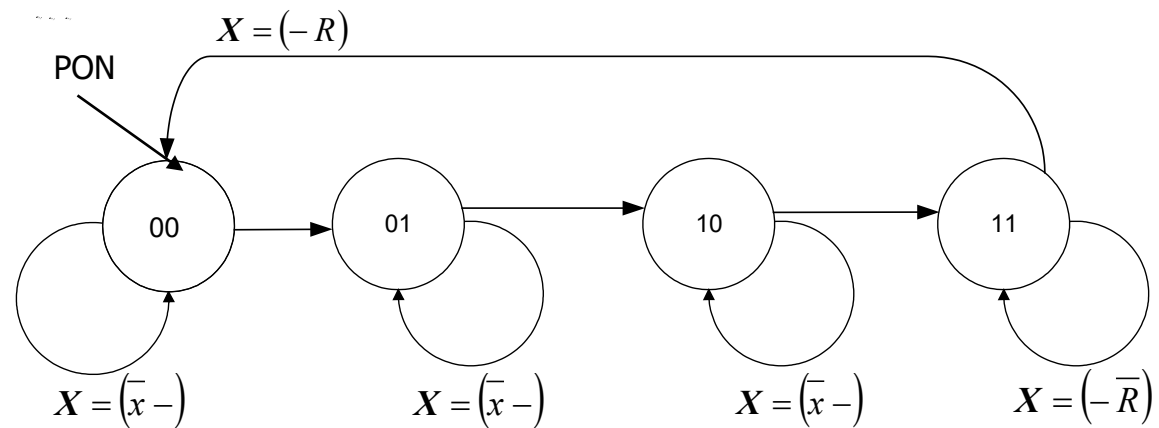
8.2 Medvedjev-Automat

■ Zustandsfolgetabelle und Automatengraph

Reduzierte Tabelle ...

X	Z	Z*
x R	$z_1 z_0$	$z_1^* z_0^*$
0 -	0 0	0 0
1 -	0 0	0 1
0 -	0 1	0 1
1 -	0 1	1 0
0 -	1 0	1 0
1 -	1 0	1 1
- 0	1 1	1 1
- 1	1 1	0 0

Die Ausgangsbelegungen sind mit den Zuständen identisch.



$$x = 1; \quad \bar{x} = 0$$

$$R = 1; \quad \bar{R} = 0$$

8.2 Medvedjev-Automat

- Charakteristische Gleichungen
 - Hinweise: Die Schaltung wird entworfen, indem man x , R , z_0 und z_1 als Eingangsvariablen und die Variablen z_0^* und z_1^* des Zustandsfolgevektors $\mathbf{Z}^*$ als Ausgänge interpretiert. Damit kann eine entsprechende kombinatorische Schaltung entworfen werden, die den Speichern vorgeschaltet, die Aufgabenstellung löst. Es wird der K-Plan verwendet. Es ist darauf zu achten, dass für die Don't-Care-Werte jeweils 0 und 1 berücksichtigt werden.

8.2 Medvedjev-Automat

■ Charakteristische Gleichungen

		z_0			
		0	1	1	0
		0	1	0	0
z_1		1	1	0	1
		1	0	0	1
		R			

$$z_0^* = \overline{R} \cdot z_1 \cdot z_0 + \overline{x} \cdot \overline{z_1} \cdot z_0 + \overline{z_0} \cdot x,$$

$$Q_A^* = \overline{R} \cdot Q_B \cdot Q_A + \overline{x} \cdot \overline{Q_B} \cdot Q_A + x \cdot \overline{Q_A}.$$

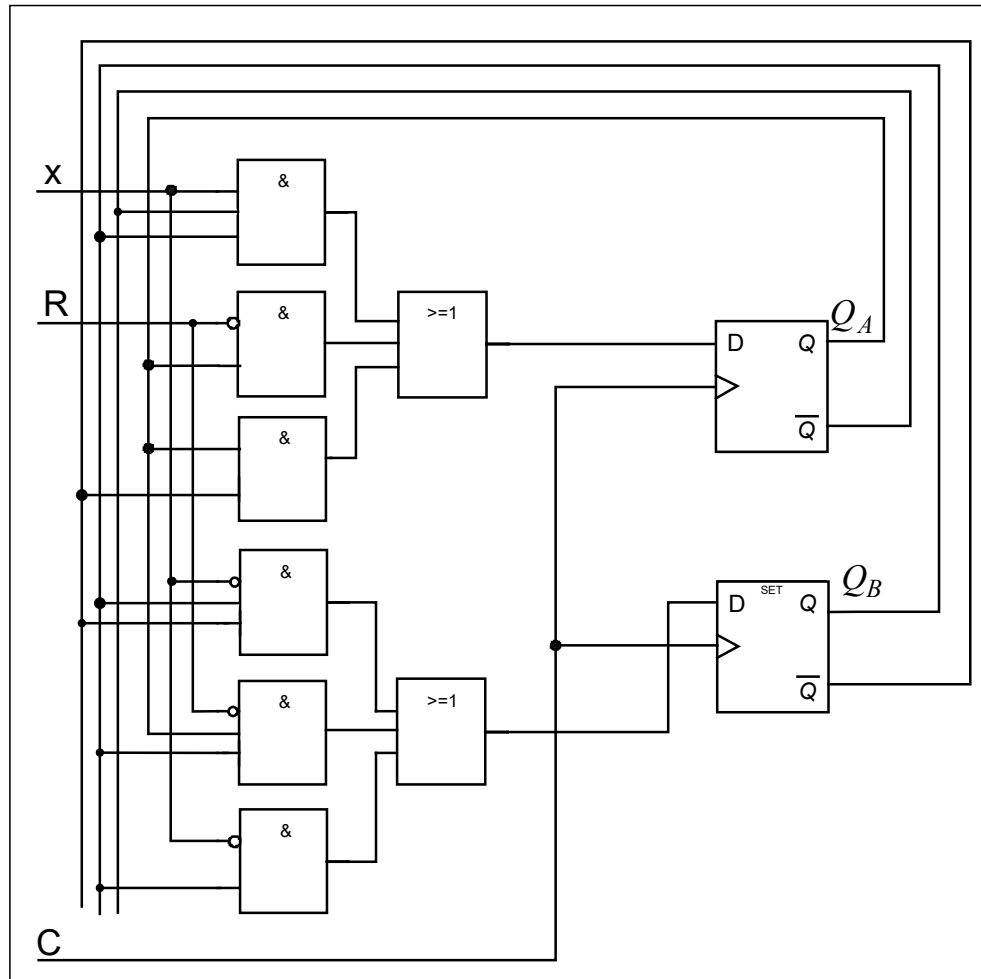
		z_0				R
		0	0	0	0	
z_1		1	1	0	1	
		1	1	0	1	
		0	1	1	0	
		R				

$$z_1^* = \overline{R} \cdot z_1 + z_1 \cdot \overline{z_0} + x \cdot \overline{z_1} \cdot z_0,$$

$$Q_B^* = \overline{R} \cdot Q_B + Q_B \cdot \overline{Q_A} + x \cdot \overline{Q_B} \cdot Q_A.$$

8.2 Medvedjev-Automat

■ Schaltung



8.3 Moore-Automat

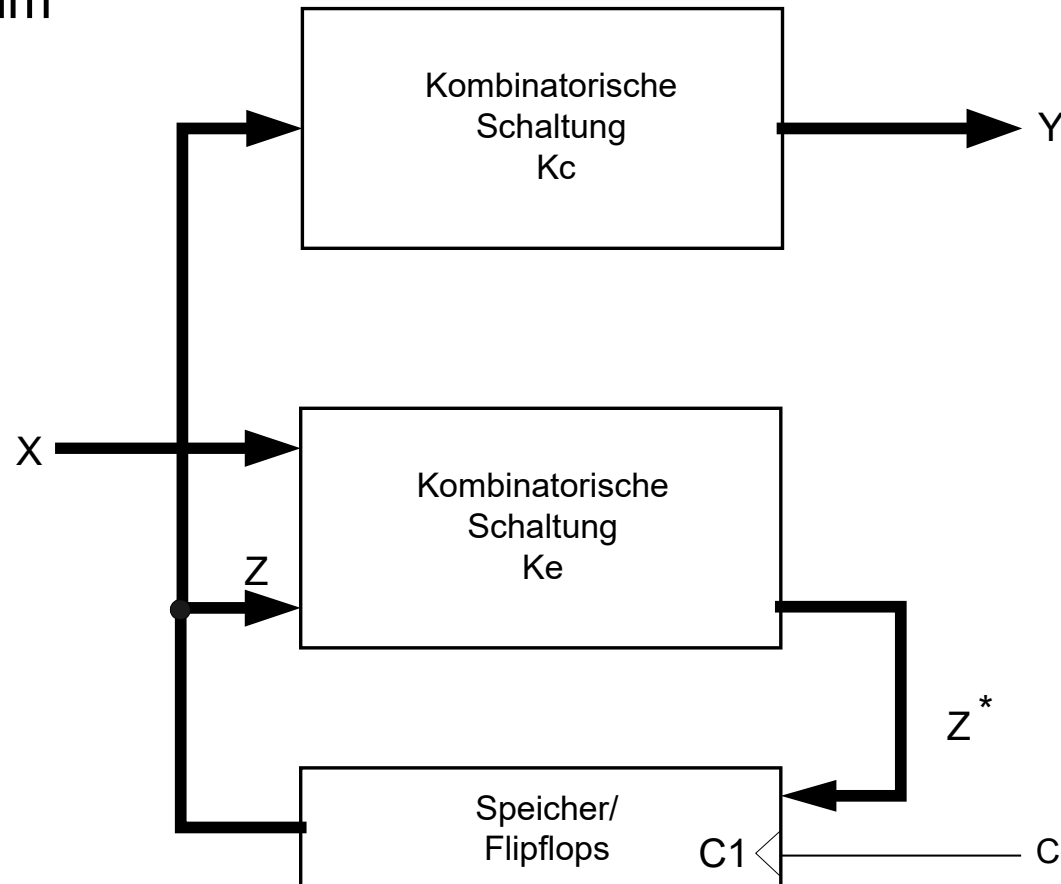
- Der Moore-Automat ist der bekannteste synchrone Automat. Für ihn gilt:

$$\mathbf{Z}^* = f(\mathbf{X}, \mathbf{Z}) \qquad \mathbf{Y}^t = f(\mathbf{Z}^t)$$

- Die Übergangsfunktionen sind von den momentanen Werten der Eingangs- und Zustandsvariablen abhängig. Die Ausgangsfunktion ist dagegen nur von den momentanen Werten der Zustandsvariablen, die über eine kombinatorische Schaltung K_c in funktionalen Zusammenhang stehen, abhängig.
- Da die Rückführung über Speicherschaltungen (z. B. D-FFs) geführt wird, ist eine Zustandsänderung nur *synchron* zum Steuertakt der D-FFs möglich, was die „*Stabilität*“ des Automaten gegenüber asynchronen Realisierungen deutlich erhöht. Die folgende Abbildung zeigt das Blockdiagramm des Moore-Automaten.

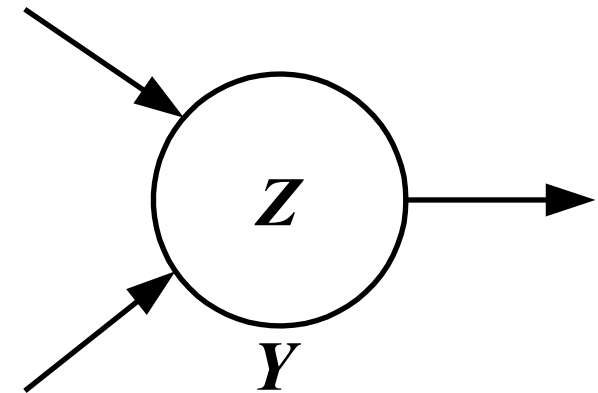
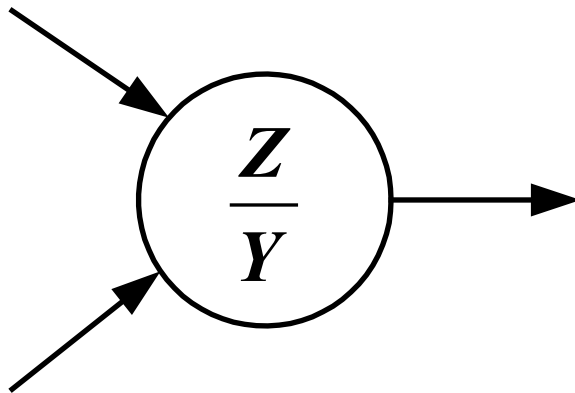
8.3 Moore-Automat

■ Blockdiagramm



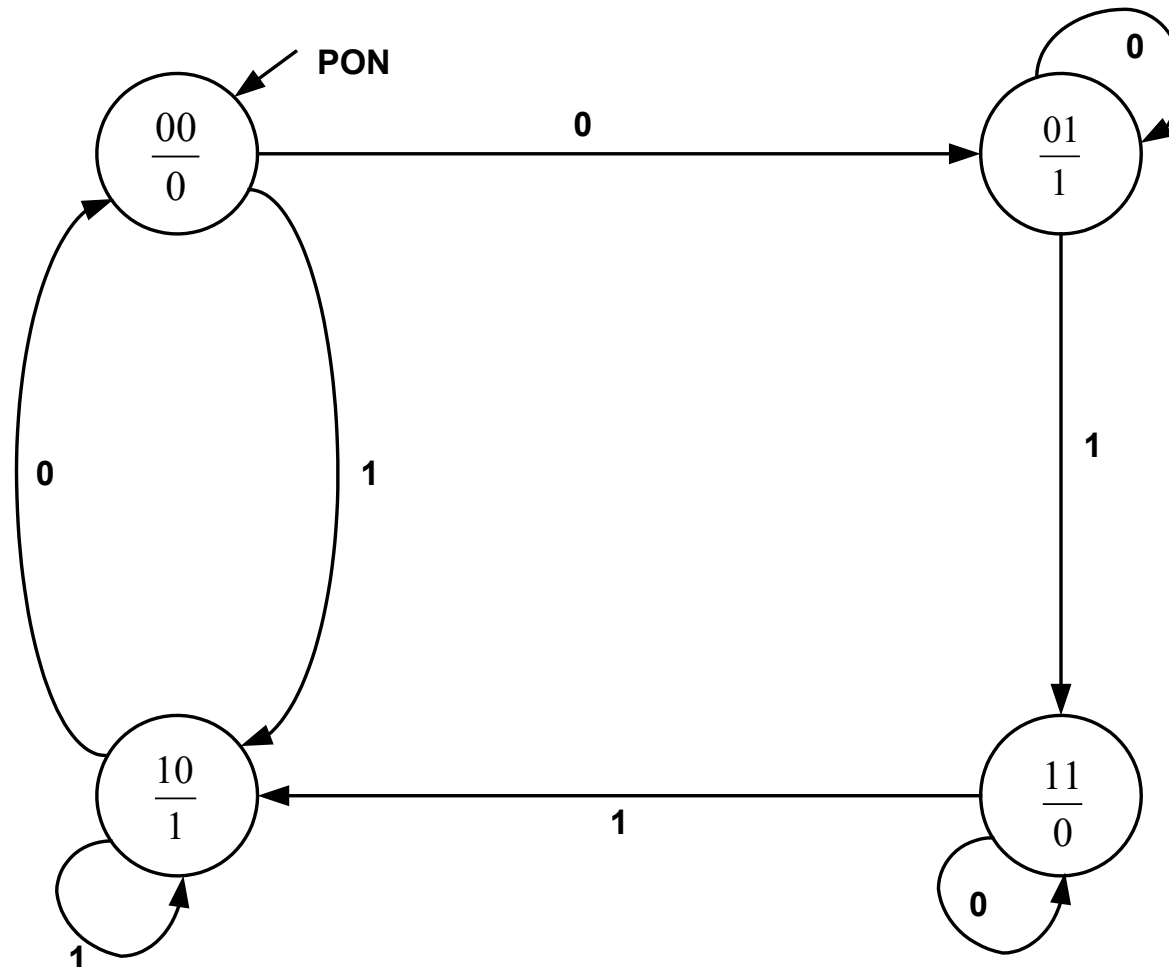
8.3 Moore-Automat

- Da der Moore-Automat synchron arbeitet, stellt der jeweilige Zustand über die kombinatorische Schaltung K_c den Ausgangsvektor dar. Er kann dem Zustand direkt zugeordnet werden. Deshalb ist es üblich den Ausgangswert entweder direkt *unter* einen Knoten oder – durch einen Strich getrennt – *in* den Knoten zu schreiben. Dabei stellt Z den jeweiligen Zustandsvektor dar.



8.3 Moore-Automat

■ Beispiel



8.3.1 Übergangsreduzierung

- In der Regel ist es günstig, die Kantengewichte in Mintermform aufzuschreiben, da sich in vielen Fällen eine Reduzierung der Gleichungen ergibt.
- Beispiel

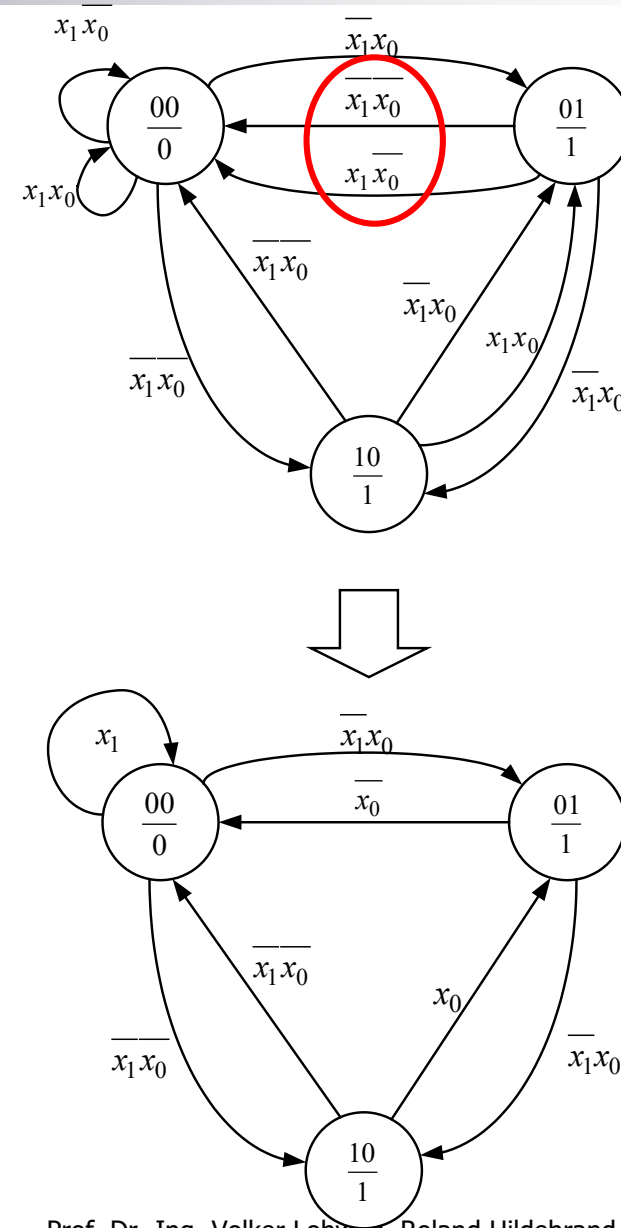
X	Z	Z*
$x_1 x_0$	$z_1 z_0$	$z_1^* z_0^*$
0 0	0 0	1 0
	0 1	0 0
	1 0	0 0
0 1	0 0	0 1
	0 1	1 0
	1 0	0 1
1 0	0 0	0 0
	0 1	0 0
	1 0	x x
1 1	0 0	0 0
	0 1	x x
	1 0	0 1

$$\overline{x_1 x_0} + x_1 \overline{x_0} = (\overline{x_1} + x_1) \overline{x_0} = \overline{x_0}$$

8.3.1 Übergangsreduzierung

- Das Vorgehen ist dabei Folgendes:

Man sucht sich alle gleichgerichteten Kanten, die von den gleichen Knoten abgehen und an gleichen Knoten enden, heraus und fasst die entsprechenden Minterme disjunktiv zusammen. Anschließend wird mit Hilfe der bekannten Absorptionsgesetze eine reduzierte Mintermform gebildet. Dieses führt dazu, dass Kanten zusammengefasst werden können – der Automatengraph wird vereinfacht.

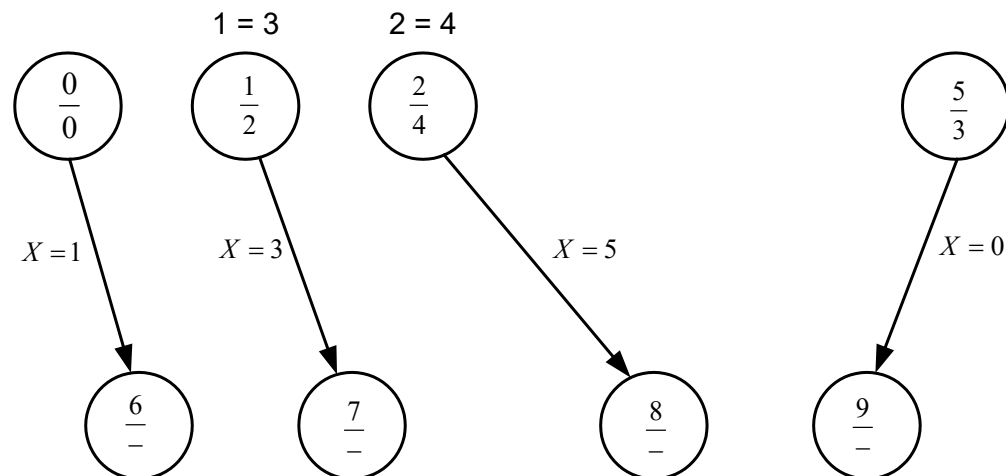
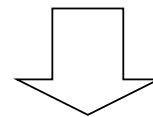
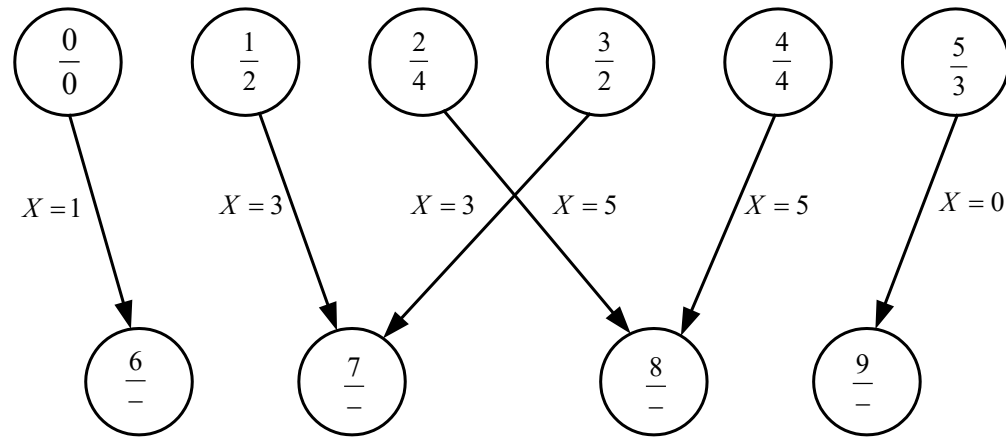


8.3.2 Zustandsreduzierung

- Neben der Reduzierung von Übergängen, ist die Reduzierung von Zuständen oftmals angezeigt. Hierdurch lässt sich die Zustandfolgetabelle in ihrer Komplexität reduzieren. Damit eine Reduktion vorgenommen werden kann, müssen die Zustände dahingehend überprüft werden, ob sie *äquivalent* sind. Diese können dann in Äquivalenzklassen zusammengefasst werden.
- **Definition 8.2** (Äquivalenzzustände)
Zwei Zustände z_A und z_B sind äquivalent, wenn zu jeder beliebigen Eingangsbelegung folgende Bedingungen zutreffen:
 - Die Folgezustände z_A^* und z_B^* sind gleich, oder als Folgezustände treten nur die betrachteten Zustände z_A und/oder z_B auf. Die Logikzustände der entsprechenden Ausgangsvariablen stimmen überein; oder ...
 - ...es tritt in einem Zustand eine Eingangsbelegung nicht auf, dann ist in diesem Fall die Äquivalenz ebenfalls gesichert.

8.3.2 Zustandsreduzierung

■ Beispiel



8.3.3 Entwurf eines Moore-Automaten

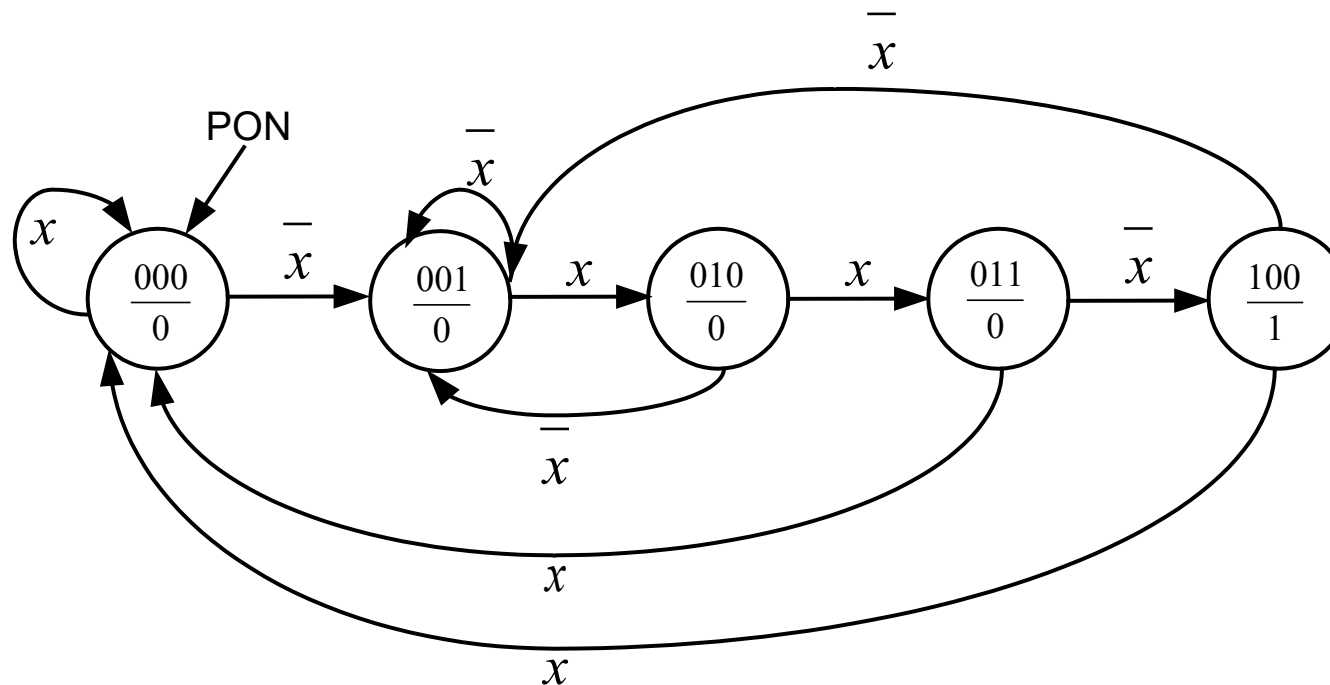
- Beispiel: Impulsfolgeerkennung

- Eine Impulsfolgeerkennung soll mit Hilfe eines Moore-Automaten realisiert werden. Die Schaltung soll eine Musterfolge von 0 und 1 erkennen. Die Folge besitzt z. B. folgende Struktur $X = (0110)$. Dabei wird die Folge mit der linken 0 als führendes Bit in die Schaltung eingetaktet. Es wird davon ausgegangen, dass x als Steuervariable des seriellen Eingangs genutzt wird. Es gilt die positive Logikvereinbarung ($x = 1$; $\bar{x} = 0$). Zunächst zeichnen wir den Automatengraphen.

8.3.3 Entwurf eines Moore-Automaten

- Beispiel: Impulsfolgeerkennung

- Automatengraph



8.3.3 Entwurf eines Moore-Automaten

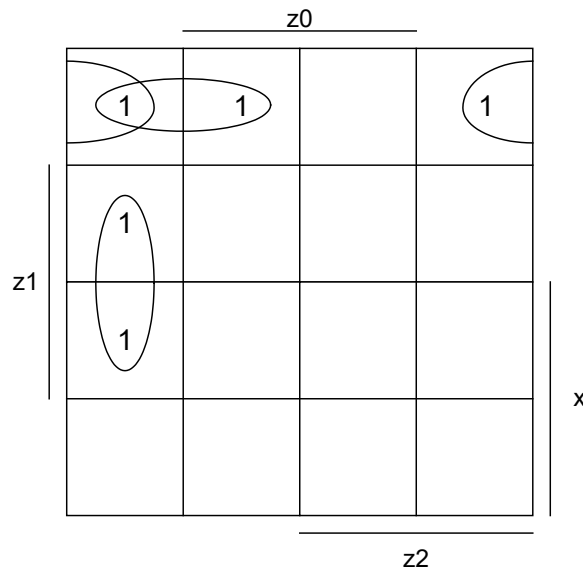
- Beispiel: Impulsfolgeerkennung
 - Zustandsfolgetabelle

X	Z	Z*	Y
x	z ₂ z ₁ z ₀	z ₂ * z ₁ * z ₀ *	y
0	0 0 0	0 0 1	0
0	0 0 1	0 0 1	0
0	0 1 0	0 0 1	0
0	0 1 1	1 0 0	0
0	1 0 0	0 0 1	1
0	1 0 1	0 0 0	0
0	1 1 0	0 0 0	0
0	1 1 1	0 0 0	0
1	0 0 0	0 0 0	0
1	0 0 1	0 1 0	0
1	0 1 0	0 1 1	0
1	0 1 1	0 0 0	0
1	1 0 0	0 0 0	1
1	1 0 1	0 0 0	0
1	1 1 0	0 0 0	0
1	1 1 1	0 0 0	0

8.3.3 Entwurf eines Moore-Automaten

- Beispiel: Impulsfolgeerkennung

- Entwurf mit D-FFs



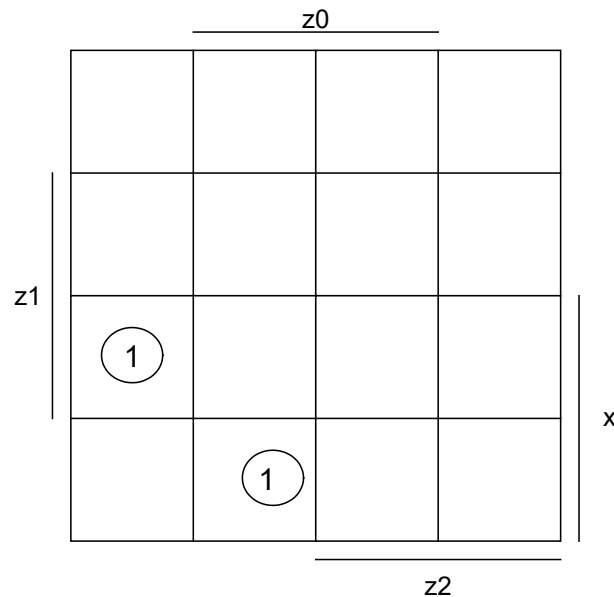
$$z_0^* = \overline{x} \cdot \overline{z_2} \cdot \overline{z_1} + \overline{z_2} \cdot \overline{z_1} \cdot \overline{z_0} + \overline{x} \cdot \overline{z_1} \cdot \overline{z_0},$$

$$Q_A^* = \overline{x} \cdot \overline{Q_C} \cdot \overline{Q_B} + \overline{Q_C} \cdot \overline{Q_B} \cdot \overline{Q_A} + \overline{x} \cdot \overline{Q_B} \cdot \overline{Q_A}.$$

8.3.3 Entwurf eines Moore-Automaten

- Beispiel: Impulsfolgeerkennung

- Entwurf mit D-FFs



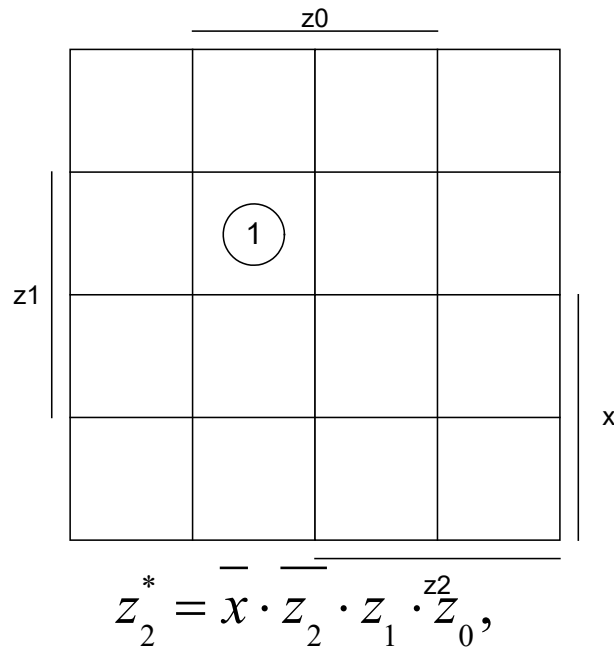
$$z_1^* = x \cdot \overline{z_2} \cdot z_1 \cdot \overline{z_0} + x \cdot \overline{z_2} \cdot \overline{z_1} \cdot z_0,$$

$$Q_B^* = x \cdot \overline{Q_C} \cdot Q_B \cdot \overline{Q_A} + x \cdot \overline{Q_C} \cdot \overline{Q_B} \cdot Q_A.$$

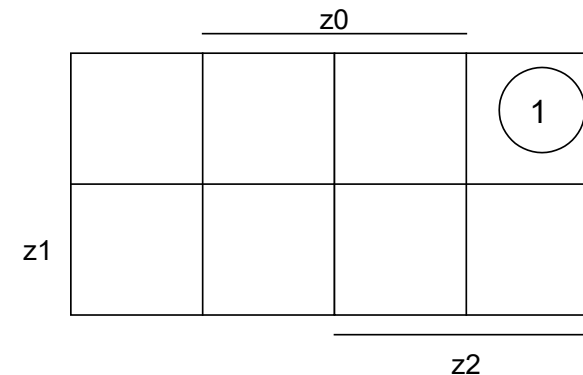
8.3.3 Entwurf eines Moore-Automaten

■ Beispiel: Impulsfolgeerkennung

■ Entwurf mit D-FFs



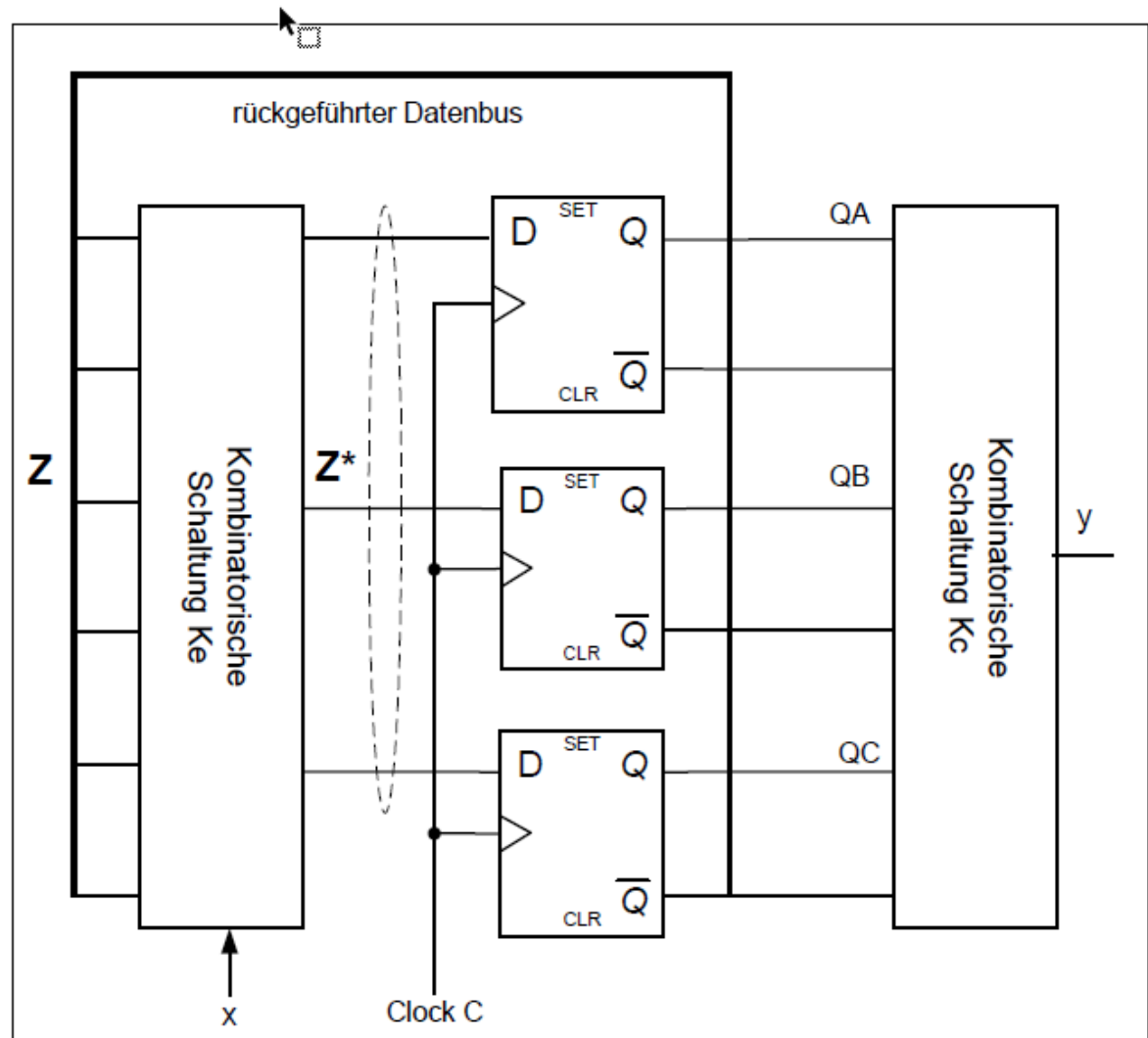
$$Q_C^* = \overline{x} \cdot \overline{Q_C} \cdot Q_B \cdot Q_A$$



$$Y = y = z_2 \cdot \overline{z_1} \cdot \overline{z_0} = Q_C \cdot \overline{Q_B} \cdot \overline{Q_A}$$

8.3.3 Entwurf eines Moore-Automaten

- Beispiel:
Impulsfolgeerkennung



8.4 Mealy-Automat

- Der Mealy-Automat ist der bekannteste synchrone Automat mit *asynchronen* Ausgängen. Für ihn gilt:

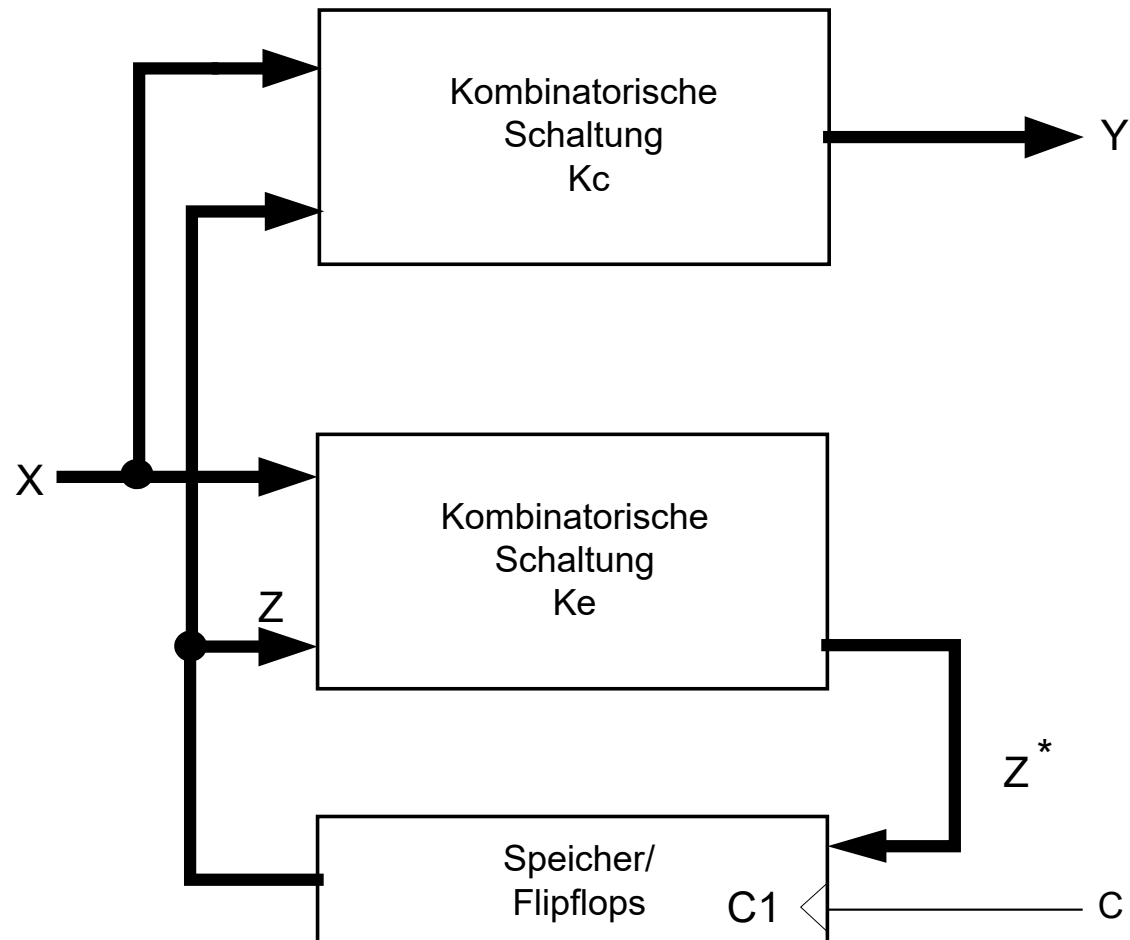
$$\mathbf{Z}^* = f(\mathbf{X}, \mathbf{Z})$$

$$\mathbf{Y}^t = f(\mathbf{Z}^t, \mathbf{X}^t)$$

- Die Übergangsfunktion ist von den momentanen Werten der Eingangs- und Zustandsvariablen abhängig. Die Ausgangsfunktion ist dagegen von den momentanen Werten der *Zustandsvariablen* und dem *Eingangsvektor*, die über eine kombinatorische Schaltung K_c im funktionalen Zusammenhang stehen, abhängig.
- Die folgende Abbildung zeigt, dass die Ausgänge *sofort* eine Änderung erfahren, wenn ein Eingangsvektor wechselt. Diese Änderung ist unabhängig vom Takt C .

8.4 Mealy-Automat

- Blockdiagramm

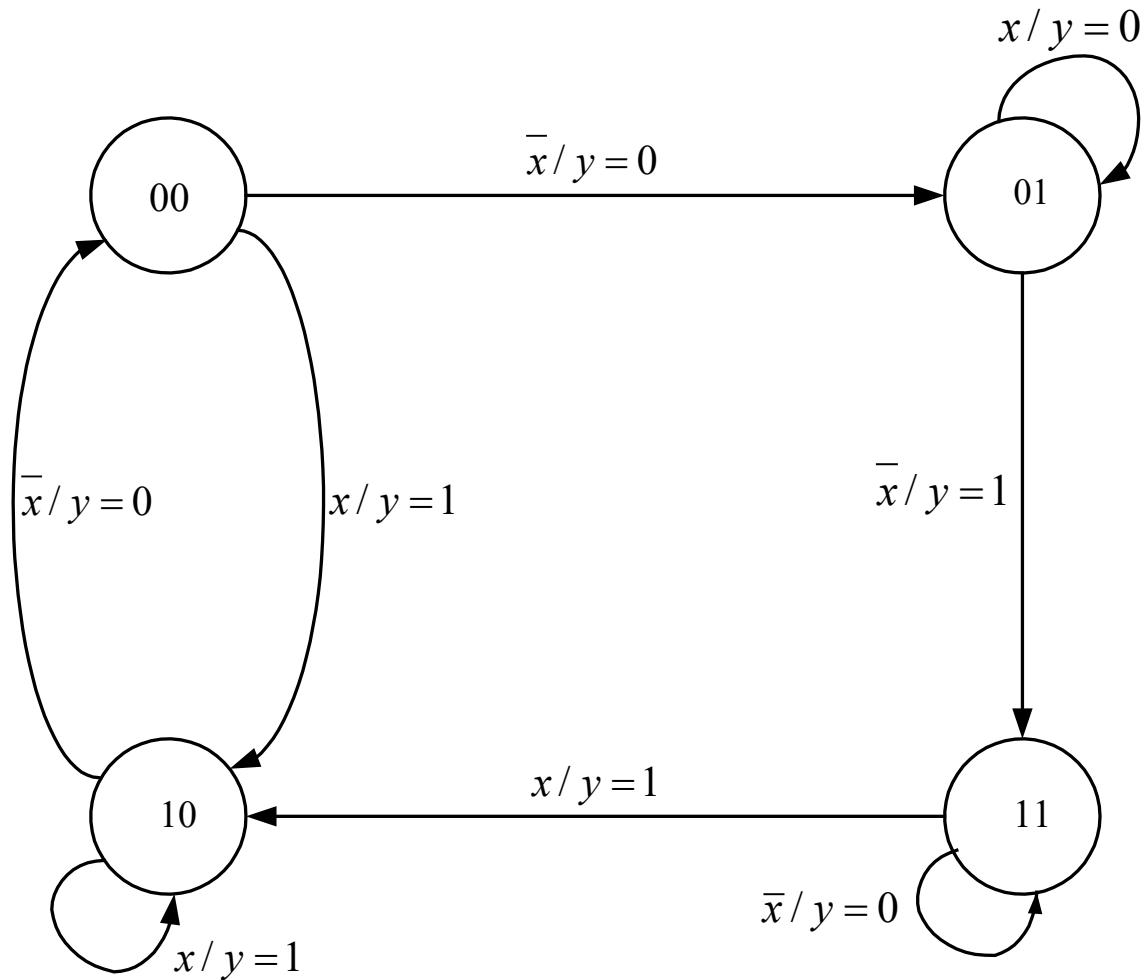


8.4 Mealy-Automat

- Da bei einem Mealy-Automaten die Ausgangsbelegungen mit den Zustandsübergängen einhergehen, können in der Regel die gleichen Ablaufsequenzen mit weniger Zuständen als bei einem Moore-Automaten realisiert werden.
- Für einen Mealy-Automaten gelten die gleichen Ansätze zur Übergangs- und Zustandsreduzierung wie bei dem Moore-Automaten. Insoweit muss in diesem Zusammenhang auf die vorherigen Abschnitte verwiesen werden.
- Für den Mealy-Automaten hat sich, ebenso wie beim Moore-Automaten, eine Symbolik für den Automatengraphen herausgebildet. Die nachfolgende Abbildung zeigt die entsprechenden Notationen. Hier wird die Ausgangsbelegung dem Übergang direkt zugeordnet.

8.4 Mealy-Automat

■ Beispiel



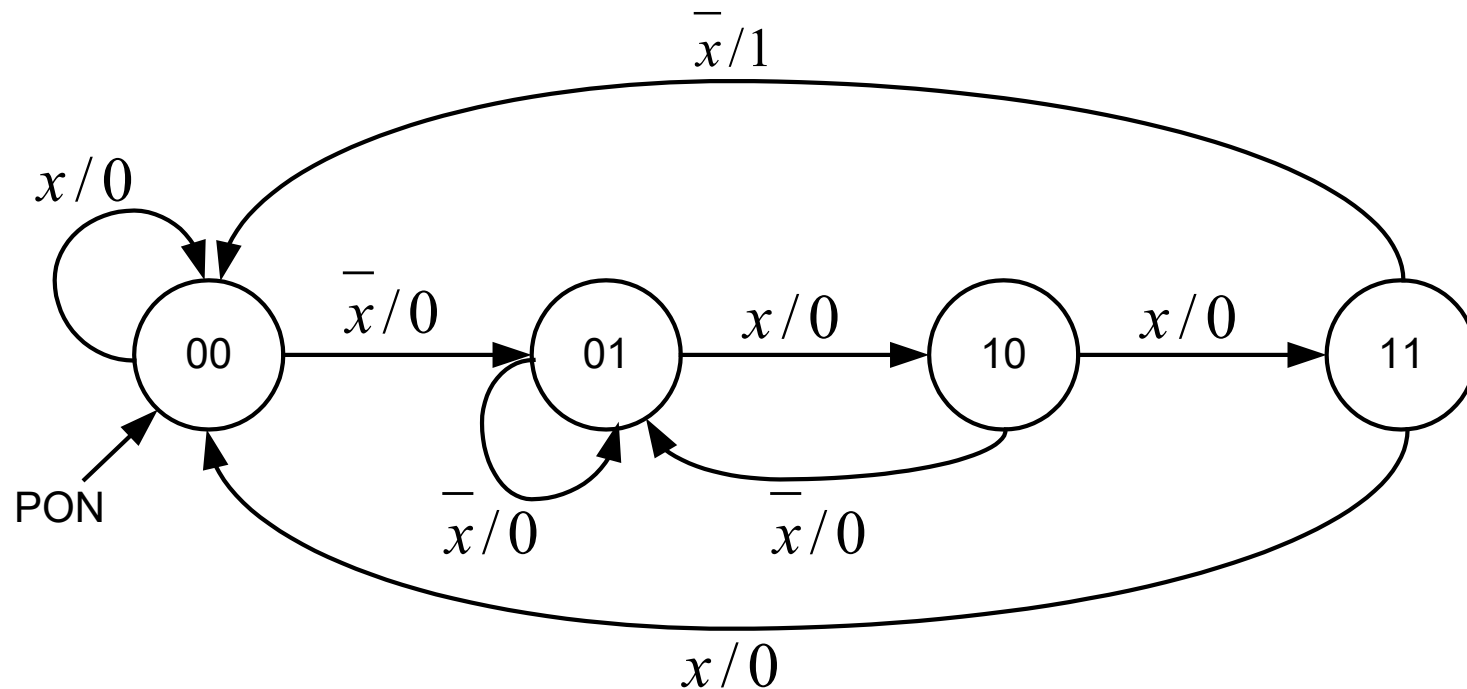
8.4.1 Entwurf eines Mealy-Automaten

- Beispiel: Impulsfolgeerkennung
- Eine Impulsfolgeerkennung soll mit Hilfe eines Mealy-Automaten realisiert werden. Die Schaltung soll eine Musterfolge von 0 und 1 erkennen. Die Folge besitzt z. B. folgende Struktur $X = (0110)$. Dabei wird die Folge mit der linken 0 als führendes Bit in die Schaltung eingetaktet. Es wird davon ausgegangen, dass die Steuervariable x des seriellen Eingangs genutzt wird (positive Logikvereinbarung).
- Zunächst zeichnen wir den Automatengraphen.

8.4.1 Entwurf eines Mealy-Automaten

- Beispiel: Impulsfolgeerkennung

- Automatengraph



8.4.1 Entwurf eines Mealy-Automaten

- Beispiel: Impulsfolgeerkennung

- Zustandsfolgetabelle

X	Z	Z*	Y
x	$z_1 z_0$	$z_1^* z_0^*$	y
0	0 0	0 1	0
0	0 1	0 1	0
0	1 0	0 1	0
0	1 1	0 0	1
1	0 0	0 0	0
1	0 1	1 0	0
1	1 0	1 1	0
1	1 1	0 0	0

8.4.1 Entwurf eines Mealy-Automaten

- Allerdings dürfte Ihnen aufgefallen sein, dass dieser Automat durch den asynchronen Ausgang problembehaftet ist.
- Wenn sie Ihr bis jetzt Erlerntes anwenden, werden Sie bemerkt haben, dass wir etwaige Verzögerungen (Latenzzeiten) unberücksichtigt gelassen haben.
- Es ist erkennbar, dass, bedingt durch entsprechende Laufzeiten, der Automat einerseits hazard-behaftet sein muss, andererseits muss die Eingangsvariable x mit dem korrekten Wert für eine bestimmte Zeit mindestens anstehen, damit der Ausgang y überhaupt schalten kann. Vergleichen Sie die Ausgangsfunktion!

8.4.1 Entwurf eines Mealy-Automaten

- Der weiter oben geschilderte Sachverhalt führt zu so genannten *vollsynchronen* Mealy-Automaten. Dabei werden direkte (kombinatorische) Verbindungen zwischen dem Eingang und dem Ausgang aufgetrennt und mit Flipflops zur Synchronisierung versehen.
- Diese Flipflops arbeiten mit dem Takt C. Dieses führt jedoch dazu, dass sich der Mealy-Automat im Wesentlichen wie ein Moore-Automat verhält!
- Es sei angemerkt, dass so genannte Äquivalenzumrechnungen zwischen den Automaten bekannt sind. An dieser Stelle muss auf die weiterführende Literatur verwiesen werden.
- In der Praxis sollte auf den Moore-Automaten zurückgegriffen werden, da die Synthese und die anschließende Analyse auf Korrektheit sich einfacher bewerkstelligen lässt.